

Day-58

Dynamic Programming (DP)

Fibonacci Series

0 1 1 2 3 5 8 13 21

$$fib(n) = fib(n-1) + fib(n-2)$$

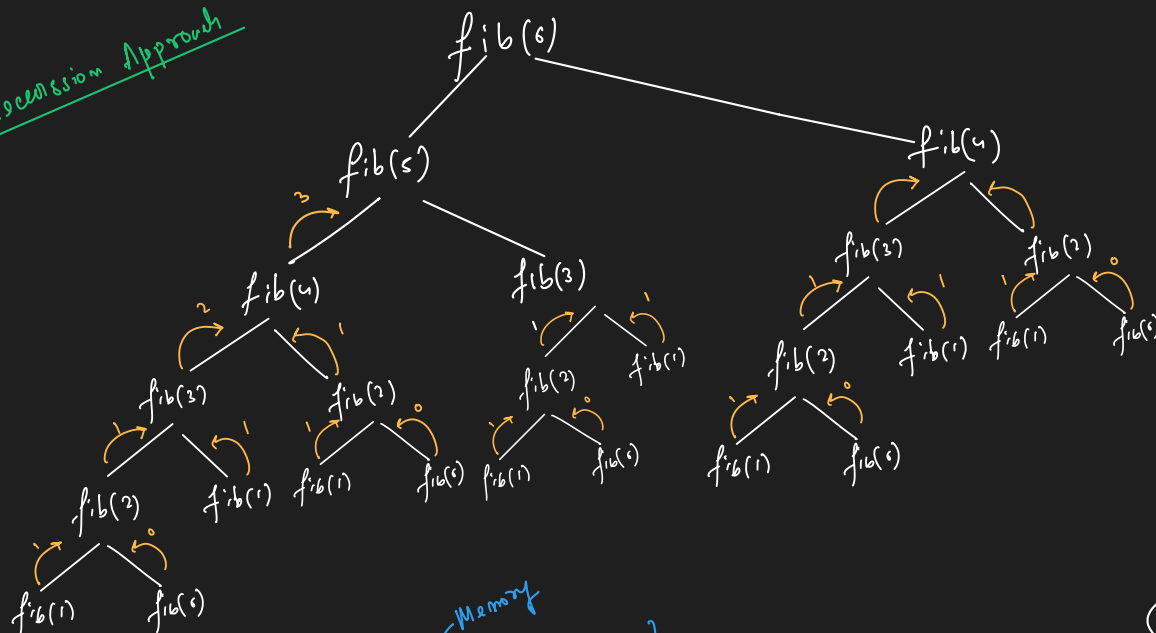
$fib(n)$?

if ($n == 0$ || $n == 1$) // base case
return n ;

return $fib(n-1) + fib(n-2)$;

}

① Recursion Approach



$T.C = (2^n)$

② Memorization Approach

Memory
[Array + Recursion]

$f[n+1] =$

0	1	2	3	4	5	6
0	1	2	3	4	5	6

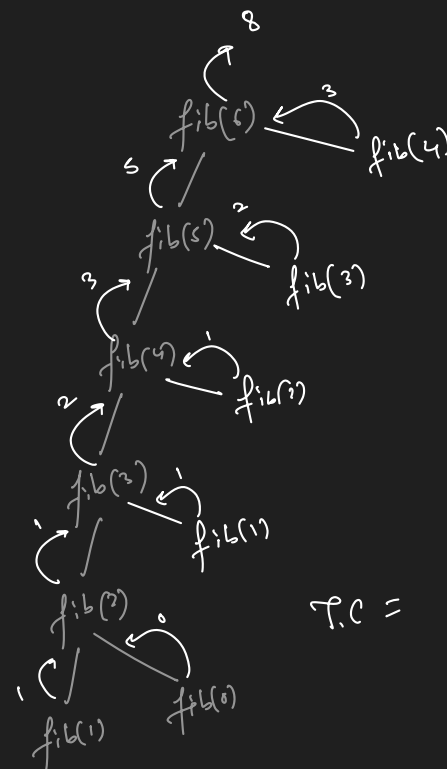
$fib(int n, int[] f)$?

if ($n == 0$ || $n == 1$) // base case
return $f[n] = n$;

if ($f[n] != 0$) // $fib(n)$ is calculated
return $f[n]$;

memorization → $f[n] = fib(n-1, f) + fib(n-2, f)$;
return $f[n]$;

}



$T.C = O(n)$

Top-Down
approach

```
public static int fibMemorisation(int n, int[] f){
    if(n==0 || n==1){
        return n;
    }

    if(f[n] != -1){
        return f[n];
    }

    f[n] = fibMemorisation(n-1, f) + fibMemorisation(n-2, f);
    return f[n];
}
```

③ Tabulation Method

DP

table

Array + for loop iteration

dp.length = n+1

dp[n+1]

dp[1] = 1

for (int i = 2; i ≤ n; i++) {

dp[i] = dp[i-1] + dp[i-2];

return dp[n];

q.c = O(n)

[Bottom-up approach]

```
public static int fibTabulation(int n){
    // Step 1 : Create DP table
    int[] dp = new int[n+1];

    // Step 2 : Initialisation
    if(n>0){
        dp[1] = 1;
    }

    // Step 3 : calculation
    for(int i=2; i<=n; i++){
        dp[i] = dp[i-1] + dp[i-2];
    }

    return dp[n];
}
```

DP : It is a technique in programming that helps to efficiently solve a class of problems that have

(i) overlapping sub-problems and (ii) optional sub-structure property.

(choices)

least, most, min, max, longest, smallest ← greedy.

Day-59

✓ Fibonacci

- ✓ Climbing Stairs
- Number Factors
- ✓ Min Jumps to reach the end
- Min Jumps with cost
- House thief

Unbounded Knapsack

- ✓ Rod Cutting
- ✓ Coin Change
- ✓ Min Coin Change
- Max Ribbon Cut

✓ 0-1 Knapsack

- ✓ Subset Sum
- Equal Subset Sum partition
- Min Subset Sum Difference
- Count of Subset Sum
- ✓ Target Sum

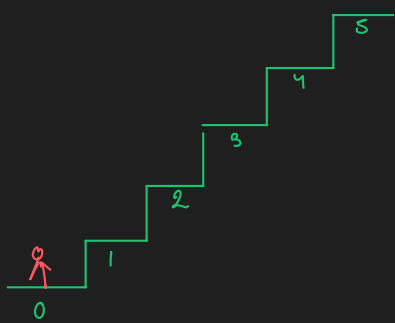
✓ LCS (Longest Common Subsequence)

- ✓ Longest Common Substring
- ✓ Edit Distance
- Box Stacking
- Longest Increasing Subsequence
- Longest Bitonic Subsequence
- ✓ Subsequence Pattern Matching
- ✓ Min Add/ Delete to convert one string to another

Catalan Number

- ✓ No. of BSTs
- N Parentheses
- Convex Polygons
- Disjoint Chords
- Catalan Triangle
- ✓ Mountain Ranges
- Dyck Words

Climbing Stair : count ways to reach the nth stair. The person can climb 1 or 2 stairs at a time.



$$f(n) = f(n-1) + f(n-2)$$

$$f(3) = f(2) + f(1)$$

dp[6]

length = 6

0	1	2	3	4	5
0	1	2	3	4	5

tabulation

initialization or Base case (for recursion and memorisation).

$$f(n) = f(n-1) + f(n-2) \rightarrow \textcircled{1} \text{ recursion} \rightarrow \textcircled{2} (2^n)$$

T.C

Recursion

```
countWays(int n) {
    if (n == 0, 1, 2)
        return n;
    return countWays(n-1) + countWays(n-2);
}
```

Imp

① 1 variable is changing

1D $\rightarrow$ dp[]

2 variable is changing

2D $\rightarrow$ dp[i][j]

Memorization

```
countWays(int n, dp[n+1]) {
    if (n == 0, 1, 2)
        return n;
    if (dp[n] != -1) // already calculated
        return dp[n];
    dp[n] = dp[n-1] + dp[n-2];
    return dp[n];
}
```

-1	-1	-1	...	...	-1
0	1	2			n

② memoizing
 $dp[i] \leftarrow$ no. of ways to reach in stair 'i'

Tabulation

```
countWays(int n) {
    dp[n+1] // step 1: table creation
    // step 2: initialization
    dp[1] = 1, dp[2] = 2
    // step 3: calculation
    for (int i = 3; i < n+1; i++) {
        dp[i] = dp[i-1] + dp[i-2];
    }
    return dp[n];
}
```

```
public static int countWays(int n){
    int[] dp = new int[n+1]; // table creation

    for(int i=1; i<n+1; i++){
        if(i <= 2){ // initialisation
            dp[i] = i;
        }else{
            dp[i] = dp[i-1] + dp[i-2]; // calculation
        }
    }

    return dp[n];
}
```

0-1 knapsack

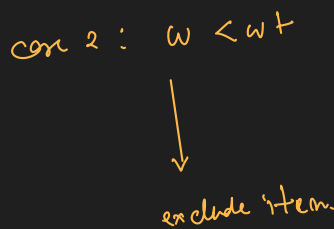
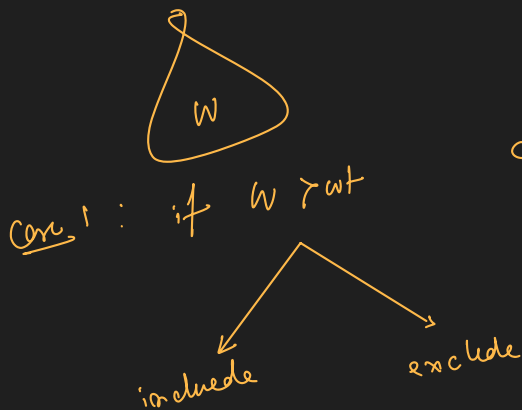
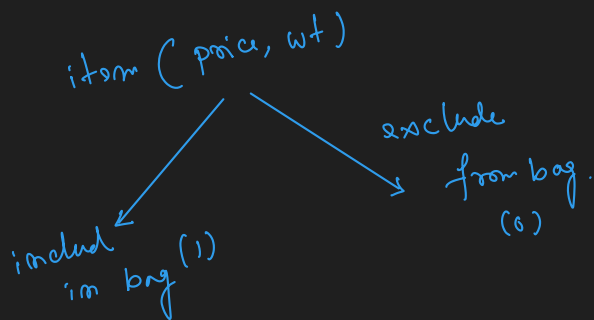
I_0, I_1, I_2, I_3, I_4
 price[] = 15, 14, 10, 45, 30
 wt[] = 2, 5, 1, 3, 4

$W = 7$



$$45 + 30 = 75$$

Max Profit = 75



Recursion

knapsack (price[], wt[], int W, int n) 7, 5

if (W == 0 || n == 0) {
 return 0;

{

if (wt <= W) {
 // include (W-wt) ← bag capacity
 (n-1) ← no. of items-

include Profit = price +
 knapsack (price, wt, W-wt, n-1)

concept

fractional knapsack (greedy)

0-1 knapsack

price[] = 15, 14, 10, 45, 30

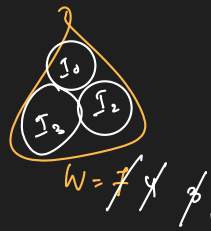
wt[] = 2, 5, 1, 3, 4

I_0, I_1, I_2, I_3, I_4

Price/wt [] = { 7.5, 2.8, 10, 15, 7.5 }

I_0, I_1, I_2, I_3, I_4

P/w	15	10	7.5	7.5	2.8
item	I_3	I_2	I_4	I_0	I_1
	✓	✓	X	✓	



$$\text{max profit} = 45 + 10 + 15 = 70$$

// exclude

$W \leftarrow \text{bag capacity}$

$(m-1) \leftarrow \text{no. of items.}$

$\text{excludeProfit} = \text{knapSack}(\text{price}, \text{wt}, W, m-1)$

return $\max(\text{includeProfit}, \text{excludeProfit})$;

}
else if $wt > W \rightarrow \text{exclude}$

$W \leftarrow \text{bag capacity}$

$(m-1) \leftarrow \text{no. of items.}$

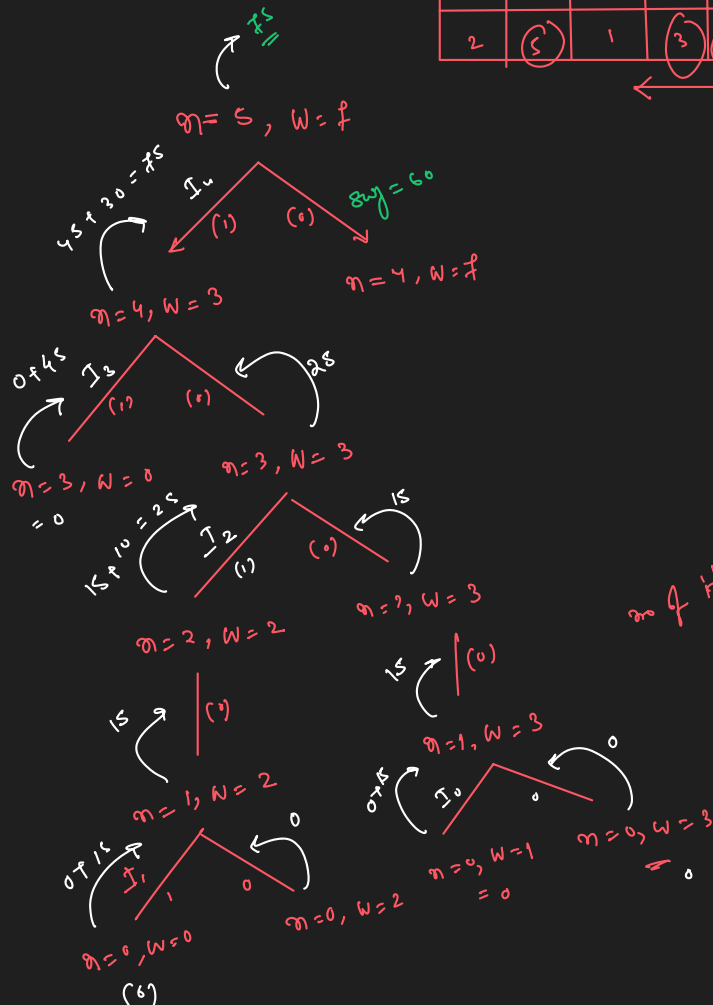
return $\text{knapSack}(\text{price}, \text{wt}, W, m-1)$;

}

I_0	I_1	I_2	I_3	I_4
15	14	10	45	30
2	5	1	3	4

}

Dry Run



Variable m, W

① $dp \rightarrow 2D ?$ $dp[m+1][w+1]$

bag capacity

	0	1	2	3	4	5	6	7
$m+1$	0							
1								
2								
3								
4								
5								

no. of items

dp sum
Memory sum

knapSack (price[], wt[], int n, int w, dp[1][1]) {

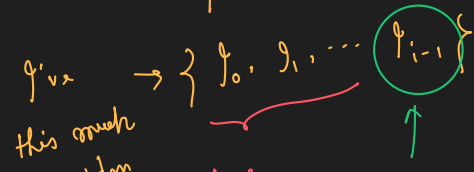
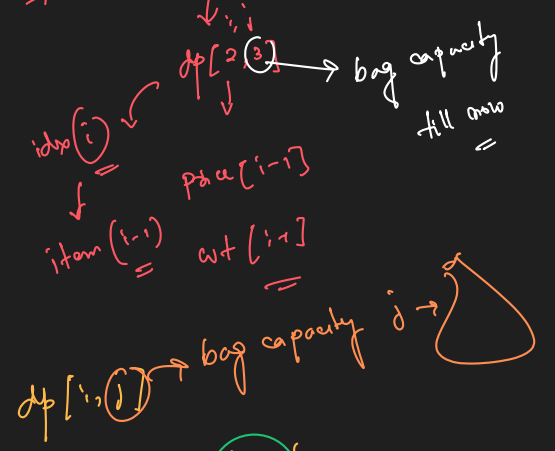
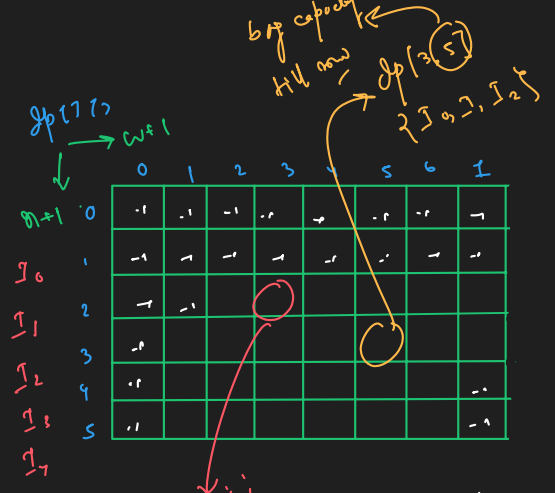
if (w == 0 || n == 0) return 0;

if (dp[n][w] != -1) { return dp[n][w]; }

if (w >= wt[n-1]) {
 incP = price[n-1] + knapSack (price, wt, n-1, w-wt[n-1]);
 excP = knapSack (price, wt, n-1, w);
 dp[n][w] = max(incP, excP);

} else {
 dp[n][w] = knapSack (price, wt, n-1, w);

return dp[n][w]; }



I'm considering this much item
 I'm checking for this item
 that profit for these much item is already been calculated
 although this is not calculated yet / only be calculated
 will be done by recursion call

```
public static int maxProfitMem(int []price, int []wt, int n, int w, int dp[][]){
    if(n == 0 || w == 0){
        return 0;
    }
    if(dp[n][w] != -1){
        return dp[n][w];
    }

    if(w >= wt[n-1]){
        int incP = price[n-1] + maxProfitMem(price, wt, n-1, w-wt[n-1], dp);
        int excP = maxProfitMem(price, wt, n-1, w, dp);

        return dp[n][w] = Math.max(incP, excP);
    }else{
        return dp[n][w] = maxProfitMem(price, wt, n-1, w, dp);
    }
}
```

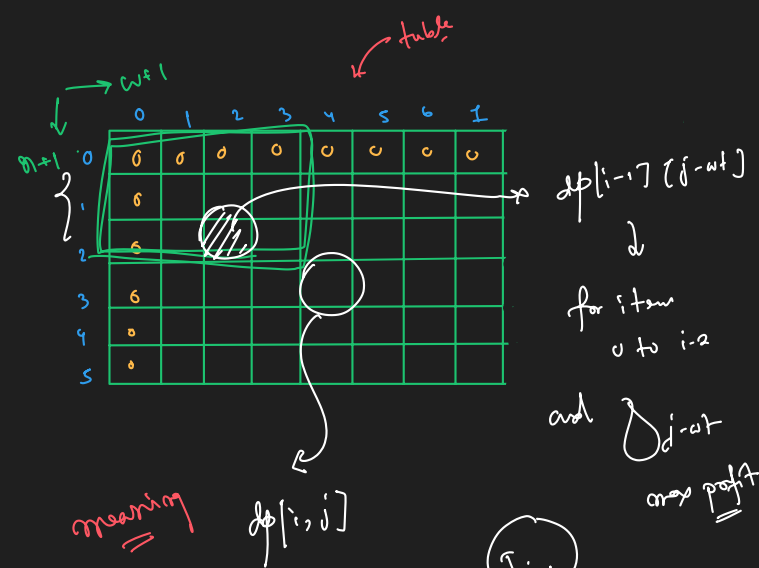
```
public static void main(String[] args) {
    int price[] = {15, 14, 10, 45, 30};
    int wt[] = {2, 5, 1, 3, 4};
    int w = 7;

    int n = price.length;

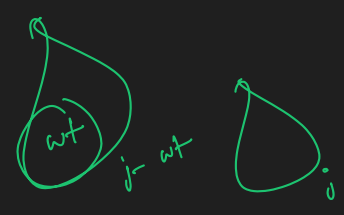
    int dp[][] = new int[n+1][w+1];
    for(int[] row : dp){
        Arrays.fill(row, -1);
    }

    System.out.println(maxProfitMem(price, wt, n, w, dp));
}
```

Tabulation
 - [bottom up approach]
 row $\rightarrow 0$ $\swarrow$ $0 = 0$ $\wedge$ $w = 0$ $\rightarrow$ $dp = 0$
 no item is consider yet $\rightarrow dp[i][j] = 0$ $\leftarrow$ initialization



// calculation
 for (int i = 1 to n+1) {
 for (j = 1 to w+1) {
 val = price[i-1]
 wt = wt[i-1]
 if (wt <= j) {



for item $I_0, I_1, \dots, I_{i-1}$
 and bag capacity = j
 the max profit store in $dp[i][j]$

$dp[i][j] = \max(\underbrace{val + dp[i-1][j-wt]}_{\text{include item}}, \underbrace{dp[i-1][j]}_{\text{exclude item}});$ is already been calculated

else $dp[i][j] = dp[i-1][j];$ // exclude

return $dp[n][w];$

```
public static int maxProfitTabular(int []price, int []wt, int n, int w){
    int dp[][] = new int[n+1][w+1];
    for(int i=1; i<n+1; i++){
        for(int j=1; j<w+1; j++){
            int iw = wt[i-1];
            if(iw<=j){
                dp[i][j] = Math.max(price[i-1] + dp[i-1][j-iw], dp[i-1][j]);
            }else{
                dp[i][j] = dp[i-1][j];
            }
        }
    }

    for(int i=0; i<=n; i++){
        for(int j=0; j<=w; j++){
            System.out.print(dp[i][j]+" ");
        }
        System.out.println();
    }

    return dp[n][w];
}
```